Ollscoil Mh Nuad
Maynooth University

Maynooth University
National University of Ireland Maynooth

SEMESTER 1

2025-2026

CS357

Software Verification

Dr. M. Llorens-Salvador, Dr. A. Mooney, Prof. R. Monahan, Dr. H. Wu

Time allowed: 2 hours

Answer at least ***three*** questions

Your mark will be based on your best ***three*** answers

All questions carry equal marks

Instructions

	Yes	No	N/A
Formulae and Tables book allowed (i.e. available on request)		**X**	
Formulae and Tables book allowed (i.e. distributed prior exam commencing)		**X**	
Statistics Tables and Formulae allowed (i.e. available on request)		**X**	
Statistics Tables and Formulae allowed (i.e. distributed prior exam commencing)		**X**	
Dictionary allowed (supplied by the student)		**X**	
Non-programmable calculator allowed		**X**	
Students required to write in and return the exam question paper		**X**	

1. [25 marks]

(a) What is a class invariant? What is the logical relationship between the post-condition in a class and the post-condition in any subclass of that class ? [8 marks]

(b) For the following lemmas, [12 marks]

- $(p \rightarrow q) \wedge r \vdash (p \rightarrow q) \wedge (p \rightarrow r)$
- $\forall x(Q(x) \rightarrow R(x)), \exists x(P(x) \wedge Q(x)) \vdash \exists x(P(x) \wedge R(x))$

1. Use natural deduction proof rules to show validity of each lemma.
2. Show relevant Coq tactics for proving each lemma.

(c) Explain the difference between satisfiability and validity of a formula. [5 marks]

2. [25 marks]

(a) For the following piece of code,

- Draw a control flow diagram. [2 marks]
- Write a Cyclone specification that can verify this program. [5 marks]

```
a=x+1; if (a-1==0) y=1;  else y=a;
```

(b) For the following piece of code,

- Draw a control flow diagram. [4 marks]
- Specify a correct loop invariant. [4 marks]
- Write a Cyclone specification that can verify this program. [10 marks]

```
i=Y;
z=0;
while (i!=0){
        i--;
        z+=X;
}
```

The precondition here is $X \geq 0 \wedge Y \geq 0$ and postcondition is $z = X \cdot Y$.

3. [25 marks]

(a) With reference to the Dafny $List$ datatype definition below: [9 marks]

i. Describe what is verified in the given lemma.

ii. Explain the difference between methods, functions and lemmas.

iii. Provide a function that appends two $List$s into one.

```
datatype List<T> = Nil | Cons(head: T, tail: List<T>)

function Length<T>(xs: List<T>): nat {
  match xs
  case Nil => 0
  case Cons(_, tail) => 1 + Length(tail)
}

function Length'<T>(xs: List<T>): nat {
  if xs == Nil then 0 else 1 + Length'(xs.tail)
}

lemma LengthLength'<T>(xs: List<T>)
  ensures Length(xs) == Length'(xs) { }
```

(b) With respect to method $M1$ in the Dafny program below: [10 marks]

i. Explain the $requires$ and $ensures$ clauses of the specification contract.

ii. Describe the role of $< T(==) >$ in the specification.

iii. Provide appropiate loop $invariant$s. Explain how these enable the Dafny verifier to verify the code.

iv. Provide a variant function. Explain how this enables the Dafny verifier to verify the code.

v. Describe and explain the expected verification error if the initialisation $i := 0$ is omitted.

vi. Describe and explain the expected verification error if the loop body statement $i := i + 1$ is replaced with $i := i * 2$.

```
method M1<T(==)>(a:array<T>, x:T) returns (r:int)
requires a != null;
ensures 0 <= r <= a.Length;
ensures r < a.Length ==> a[r] == x;
ensures forall j :: 0 <= j < r ==> a[j] != x;
{
  var i := 0;
  while (i < a.Length && a[i] != x)
  {
     i:= i+1;
  }
  return i;
}
```

(c) Explain how the Dafny verifier uses Hoare logic to generate verification conditions. Provide the precondition and postcondition of the loop body in method $M1$. State which Hoare rules are used to generate them. Explain how the rules are applied. [6 marks]

4. [25 marks]

(a) Describe how the Formal Requirements Elicitation Tool (FRET) supports model checking. In particular, your answer should describe the FRETish language, the formalisation of requirements and at least two verification tools which integrate with FRET. Provide an example requirement expressed in FRETish to support your answer. [8 marks]

(b) Describe the difference between *Software Testing* and *Model Checking*. Explain the main obstacle encountered by model checking algorithms. Use an example to support your answer. [7 marks]

(c) Specify the following properties in LTL. Explain your answers. [10 marks]

- Always, if an email is sent through the network, then it will eventually be delivered.
- If a car is parked and the meter has expired, then at the next moment it will be ticketed.
- Always, after the machine gets a coin and the user press a button, it gives coffee or tea.
- Always, after pressing a button the machine will serve coffee and then tea immediately afterward.
- The gate remains closed until the train leaves the crossing.